Spectral Word Embeddings

In the preceding articles, we developed n-gram language models that assign probabilities to word sequences by counting co-occurrences in a corpus. Those models treat each word as an atomic symbol—"cat" and "dog" are as different from each other as "cat" and "telescope." But this misses something fundamental: words have *relationships*. Some words are similar in meaning, some are opposites, some share grammatical roles. If we could represent words in a way that captures these relationships, we could build more powerful models that generalize better from limited data.

This article introduces **word embeddings**—the idea of representing words as vectors in a continuous space, where geometric relationships between vectors reflect linguistic relationships between words. We develop a specific classical technique for constructing such embeddings: **spectral embedding via the graph Laplacian**, which uses the distributional context of words in a corpus to induce vector representations. This approach predates the neural embedding methods (Word2Vec, ELMo, etc.) that we will cover in future articles, but it beautifully illustrates the core ideas and connects naturally to the count-based methods we have already studied.

Why Represent Words as Vectors?

The Problem with Discrete Representations

In our n-gram models, words are elements of a finite vocabulary $V = \{w^{(1)}, w^{(2)}, \ldots, w^{(|V|)}\}$. Each word is simply an index—an integer from 1 to $|V|$. This **discrete** representation has a fundamental limitation: there is no notion of similarity between words. The indices are arbitrary. The fact that "cat" is word 3,417 and "dog" is word 8,902 tells us nothing about their relationship.

This matters practically. Suppose our language model has learned from training data that "the cat sat on the mat" is a likely sentence. With discrete representations, this knowledge provides *zero* information about "the dog sat on the mat"—the model treats "dog" as completely unrelated to "cat." Every word must be learned about independently, which means we need enormous amounts of data to cover the combinatorial space of possible word sequences.

One-Hot Encoding

The most natural way to convert discrete symbols into vectors is **one-hot encoding**: represent each word as a vector in $\mathbb{R}^{|V|}$ with a 1 in the position corresponding to its index and 0 everywhere else.

For a vocabulary of 4 words {cat, dog, mat, sat}:

$$\text{cat} = \begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \quad \text{dog} = \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \quad \text{mat} = \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \quad \text{sat} = \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$$

One-hot vectors are orthogonal to each other: for any two distinct words $w^{(i)}$ and $w^{(j)}$, their inner product is zero:

$$\langle \mathbf{e}_i, \mathbf{e}_j \rangle = 0 \quad \text{for } i \neq j$$

This means every pair of words is equidistant from every other pair. "Cat" is as far from "dog" as it is from "telescope." The geometry of the space carries no linguistic information whatsoever.

One-hot representations also suffer from a **dimensionality problem**. With a vocabulary of 50,000 words, each word is a vector in $\mathbb{R}^{50000}$. These vectors are extremely sparse (99.998% zeros) and live in a space far too large for the amount of information they encode.

Dense Vector Representations: Embeddings

A **word embedding** maps each word to a dense vector in a much lower-dimensional space $\mathbb{R}^d$, where $d \ll |V|$ (typically d ranges from 50 to 300):

$$\phi : V \to \mathbb{R}^d$$

The key property is that this mapping is *not arbitrary*—it is constructed so that the geometry of the vector space reflects linguistic structure:

- **Similar words are nearby**: $\|\phi(\text{cat}) - \phi(\text{dog})\|$ is small
- **Dissimilar words are far apart**: $\|\phi(\text{cat}) - \phi(\text{telescope})\|$ is large
- **Relationships are directions**: The vector from $\phi(\text{king})$ to $\phi(\text{queen})$ may be similar to the vector from $\phi(\text{man})$ to $\phi(\text{woman})$

This is a profound shift. Instead of treating words as arbitrary symbols, we represent them as points in a geometric space where distance, angle, and direction all carry meaning. The space has **structure**—and that structure encodes linguistic knowledge.

What Embeddings Buy Us

Dense vector representations provide several concrete advantages over discrete or one-hot representations.

Generalization through similarity. If a model has learned something about "cat," it can transfer that knowledge to "dog" because their vectors are nearby. This addresses the sparsity problem that plagues n-gram models: instead of needing to observe every possible word combination, the model can generalize from observed combinations to similar ones.

Dimensionality reduction. Instead of $|V|$-dimensional one-hot vectors, we work with d-dimensional embeddings where d is typically 100-300. This makes downstream computation tractable and forces the representation to capture the most important distinctions.

Geometric structure. The embedding space has a rich geometry. We can measure similarity via cosine similarity or Euclidean distance. We can find nearest neighbors. We can perform arithmetic on word vectors. We can cluster words into groups. We can project the space to 2D or 3D for visualization. This geometry provides tools for exploring and understanding language that simply don't exist with discrete representations.

Input to machine learning models. Most machine learning algorithms—linear classifiers, neural networks, SVMs—operate on continuous vectors. Embeddings provide a natural way to feed words into these models. A model that takes word embeddings as input can exploit the geometric structure: it can learn a decision boundary that applies to regions of the embedding space rather than to individual words.

The Distributional Hypothesis

Words of a Feather Flock Together

How should we decide which words get similar vectors? We need a principle for measuring word similarity. The answer comes from a deep insight in linguistics:

> *"You shall know a word by the company it keeps." — J. R. Firth (1957)*

This is the **distributional hypothesis**: words that appear in similar contexts tend to have similar meanings. "Cat" and "dog" are similar because they appear in similar linguistic environments—after "the," before "sat," as objects of "pet," and so on. "Cat" and "telescope" appear in very different contexts.

The distributional hypothesis connects naturally to the n-gram models we have already studied. N-gram models are fundamentally about context: they estimate the probability of a word given the surrounding words. The distributional hypothesis says that this contextual information—the patterns of which words appear near which other words—contains enough signal to determine word similarity.

This principle predates neural networks by decades. It was articulated by Firth in the 1950s and developed further by Harris (1954) and others in the structuralist linguistics tradition. The idea that meaning can be inferred from usage patterns, without any reference to the external world, is both powerful and somewhat surprising.

Defining Context

To make the distributional hypothesis precise, we need a formal definition of **context**. Given a word w in a corpus, its context consists of the words that appear near it. There are many ways to formalize "near":

Positional context features. Define positional features w_i, where i is a relative position. For a target word w, the feature w_i denotes the word appearing i positions away (positive for right, negative for left).

Example. Consider the sentence:

> *"Computer Science is no more about computers than astronomy is about telescopes."*

For the target word "computers" (position 7), some positional context features are:

- $w_1 = \text{than}$ (one word to the right)
- $w_{-1} = \text{about}$ (one word to the left)
- $w_{-2} = \text{more}$ (two words to the left)

Paired context features. We can also define tuple features $w_{i,j}$, which capture the pair of words at relative positions i and j simultaneously:

- $w_{-4,2} = (\text{is}, \text{astronomy})$ (the words 4 to the left and 2 to the right)

These paired features capture richer patterns—they encode which words co-occur in specific configurations around the target word.

The full context set. The complete context of a word w across an entire corpus is the union of all observed context features:

$$C(w) = \{w_i \mid i \in \mathbb{Z}, i \neq 0\} \cup \{w_{i,j} \mid i, j \in \mathbb{Z},\, i \neq j,\, i \neq 0,\, j \neq 0\}$$

In practice, we restrict the context window to a fixed size (e.g., $|i| \leq 5$) and aggregate features across all occurrences of w in the corpus.

From Context to Similarity

Measuring Word Similarity

Given context sets for every word, we can measure how similar two words are by how much their contexts overlap. If "cat" and "dog" appear in many of the same contexts, they are similar. If "cat" and "telescope" share few contexts, they are dissimilar.

A natural measure for comparing two sets is the **Dice coefficient**, which we use here as our similarity function:

$$D(w^{(i)}, w^{(j)}) = 2 \times \frac{|C(w^{(i)}) \cap C(w^{(j)})|}{|C(w^{(i)})| + |C(w^{(j)})|}$$

The numerator counts the context features shared by both words. The denominator normalizes by the total number of features each word has, so that words with large and small context sets can be compared fairly. The factor of 2 ensures $D \in [0, 1]$: if the two words have identical context sets, $D = 1$; if they share no contexts, $D = 0$.

Why the Dice coefficient? Other set similarity measures could be used—the Jaccard index, cosine similarity on count vectors, pointwise mutual information. The Dice coefficient has the advantage of being symmetric, bounded, and easy to compute. It gives slightly more weight to shared features than the Jaccard index (which divides by $|A \cup B|$ instead of $|A| + |B|$), making it somewhat more sensitive to overlap.

The Similarity Matrix

Computing $D(w^{(i)}, w^{(j)})$ for every pair of words in the vocabulary produces a **similarity matrix** $W \in \mathbb{R}^{|V| \times |V|}$:

$$W_{ij} = D(w^{(i)}, w^{(j)})$$

This matrix has several important properties:

- **Symmetric**: $W_{ij} = W_{ji}$ (the Dice coefficient is symmetric)
- **Non-negative**: $W_{ij} \geq 0$ (set overlaps are non-negative)
- **Diagonal entries equal 1**: $W_{ii} = 1$ (every word has perfect overlap with itself)
- **Off-diagonal entries in** $[0, 1]$: Bounded similarity scores

For a vocabulary of 5 words, the matrix looks like:

	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	$w^{(4)}$	$w^{(5)}$
$w^{(1)}$	1	$D(w^{(1)}, w^{(2)})$	$D(w^{(1)}, w^{(3)})$	$D(w^{(1)}, w^{(4)})$	$D(w^{(1)}, w^{(5)})$
$w^{(2)}$	$D(w^{(2)}, w^{(1)})$	1	$D(w^{(2)}, w^{(3)})$	$D(w^{(2)}, w^{(4)})$	$D(w^{(2)}, w^{(5)})$
$w^{(3)}$	$D(w^{(3)}, w^{(1)})$	$D(w^{(3)}, w^{(2)})$	1	$D(w^{(3)}, w^{(4)})$	$D(w^{(3)}, w^{(5)})$
$w^{(4)}$	$D(w^{(4)}, w^{(1)})$	$D(w^{(4)}, w^{(2)})$	$D(w^{(4)}, w^{(3)})$	1	$D(w^{(4)}, w^{(5)})$
$w^{(5)}$	$D(w^{(5)}, w^{(1)})$	$D(w^{(5)}, w^{(2)})$	$D(w^{(5)}, w^{(3)})$	$D(w^{(5)}, w^{(4)})$	1

This matrix encodes all pairwise word similarities. But it is $|V| \times |V|$—for a vocabulary of 50,000 words, it has 2.5 billion entries. We need a way to extract a compact, low-dimensional representation from this matrix.

The Word Similarity Graph

From Matrix to Graph

The similarity matrix W has a natural interpretation as the **adjacency matrix** of a weighted graph. Define a graph $G = (V, E)$ where:

- **Vertices**: The words $w^{(1)}, w^{(2)}, \ldots, w^{(|V|)}$
- **Edges**: Between every pair of words, with weight W_{ij}

This is a **complete weighted graph**—every pair of vertices is connected, and the edge weight encodes how similar the two words are. Strong edges connect similar words ("cat"–"dog"), weak edges connect dissimilar words ("cat"–"telescope").

Interpreting similarity as a graph is more than a metaphor. It lets us bring the powerful tools of **spectral graph theory** to bear on the problem of word representation. In particular, it connects word embedding to a well-studied problem: finding a low-dimensional representation of the vertices of a graph that preserves its structure.

What the Graph Captures

Think of the word similarity graph as a landscape. Words that share many contexts are connected by strong edges and form tightly knit clusters. Nouns cluster with nouns, verbs with verbs, and within those broad categories, finer structure emerges: animals cluster together, cooking verbs cluster together, and so on.

The topology of this graph—which clusters exist, how they connect, what the boundaries look like—encodes the structure of the language as reflected in the corpus. Our goal is to embed this graph into a low-dimensional Euclidean space in a way that preserves as much of this structure as possible.

The Graph Laplacian

The Degree Matrix

Before defining the Laplacian, we need the **degree matrix**. For a weighted graph with adjacency matrix W, the **degree** of vertex i is the sum of all edge weights incident to it:

$$d_i = \sum_{j=1}^{|V|} W_{ij}$$

The degree matrix D is the diagonal matrix with these degrees:

$$D = \text{diag}(d_1, d_2, \ldots, d_{|V|})$$

That is, $D_{ii} = d_i$ and $D_{ij} = 0$ for $i \neq j$. The degree of a word measures how "connected" it is to other words in the vocabulary—how much total context overlap it has. Common words with diverse usage patterns tend to have high degree; rare or specialized words tend to have low degree.

Definition of the Graph Laplacian

The **(unnormalized) graph Laplacian** is defined as:

$$L = D - W$$

This is an $|V| \times |V|$ matrix. Its entries are:

$$L_{ij} = \begin{cases} d_i & \text{if } i = j \\ -W_{ij} & \text{if } i \neq j \end{cases}$$

The Laplacian encodes the graph structure in a form amenable to spectral analysis. It is the discrete analog of the Laplace operator from calculus (hence the name), which measures how a function deviates from its local average.

Key Properties of the Laplacian

The graph Laplacian has several properties that make it useful for embedding:

Symmetric and positive semi-definite. Since W is symmetric, $L = D - W$ is symmetric. Moreover, for any vector $\mathbf{f} \in \mathbb{R}^{|V|}$:

$$\mathbf{f}^\top L \mathbf{f} = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$$

This identity is fundamental. It says that the **quadratic form** $\mathbf{f}^\top L \mathbf{f}$ measures the total "variation" of $\mathbf{f}$ over the graph—how much $\mathbf{f}$ changes across edges, weighted by edge strength. A function that assigns similar values to strongly connected vertices will have small $\mathbf{f}^\top L \mathbf{f}$; a function that assigns very different values to strongly connected vertices will have large $\mathbf{f}^\top L \mathbf{f}$.

Proof of the quadratic form identity:

$$\mathbf{f}^\top L \mathbf{f} = \mathbf{f}^\top D \mathbf{f} - \mathbf{f}^\top W \mathbf{f} = \sum_i d_i f_i^2 - \sum_{i,j} W_{ij} f_i f_j$$

$$= \sum_i \left(\sum_j W_{ij} \right) f_i^2 - \sum_{i,j} W_{ij} f_i f_j = \sum_{i,j} W_{ij} f_i^2 - \sum_{i,j} W_{ij} f_i f_j$$

$$= \frac{1}{2} \left(\sum_{i,j} W_{ij} f_i^2 - 2 \sum_{i,j} W_{ij} f_i f_j + \sum_{i,j} W_{ij} f_j^2 \right) = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$$

where the last step uses the symmetry $W_{ij} = W_{ji}$ to combine terms.

Since $W_{ij} \geq 0$ and $(f_i - f_j)^2 \geq 0$, we have $\mathbf{f}^\top L\mathbf{f} \geq 0$ for all $\mathbf{f}$, confirming positive semi-definiteness.

Zero eigenvalue. The constant vector $\mathbf{1} = (1, 1, \ldots, 1)^\top$ is always an eigenvector of L with eigenvalue 0:

$$L\mathbf{1} = (D - W)\mathbf{1} = D\mathbf{1} - W\mathbf{1}$$

Since $(W\mathbf{1})_i = \sum_j W_{ij} = d_i = (D\mathbf{1})_i$, we get $L\mathbf{1} = \mathbf{0}$.

This makes intuitive sense: a constant function has zero variation over the graph, so $\mathbf{1}^\top L\mathbf{1} = 0$.

Eigenvalue spectrum. Since L is positive semi-definite, all eigenvalues are non-negative: $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{|V|}$. The number of zero eigenvalues equals the number of connected components in the graph. For a single connected component (which is our case, since the graph is complete), $\lambda_1 = 0$ and $\lambda_2 > 0$.

The Normalized Graph Laplacian

In practice, we often use the **normalized graph Laplacian** instead of the unnormalized version. There are two common normalizations:

Symmetric normalized Laplacian:

$$L_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}$$

Random walk normalized Laplacian:

$$L_{\text{rw}} = D^{-1} L = I - D^{-1} W$$

The matrix $D^{-1}W$ is the **transition matrix** of a random walk on the graph: $(D^{-1}W)_{ij} = W_{ij}/d_i$ is the probability of walking from vertex i to vertex j, where edge weights determine transition probabilities.

Why normalize? The unnormalized Laplacian is biased by vertex degree: high-degree vertices (common words) dominate the spectrum. Normalization corrects for this by dividing each row by the vertex degree, ensuring that the resulting embedding is not distorted by word frequency.

For spectral word embeddings, we use the generalized eigenproblem corresponding to the random walk normalization, as described in the next section.

Spectral Embedding

The Optimization Problem

We want to find a map $\phi : V \to \mathbb{R}^k$ that assigns each word a k-dimensional vector, such that **similar words get similar vectors**. How do we formalize "similar words get similar vectors"?

Consider a single coordinate function $f : V \to \mathbb{R}$ that assigns a real number to each word. We want $f(i) \approx f(j)$ whenever W_{ij} is large—that is, whenever words i and j are similar. The quadratic form of the Laplacian measures exactly this:

$$\mathbf{f}^\top L \mathbf{f} = \frac{1}{2} \sum_{i,j} W_{ij} (f_i - f_j)^2$$

Minimizing this quantity finds a function f that varies as little as possible across edges of the graph. But the trivial minimum is $f = \text{constant}$ (which we already know is the eigenvector with eigenvalue 0). To get a non-trivial solution, we add a normalization constraint.

The spectral embedding solves:

$$\min_{\mathbf{f}} \ \mathbf{f}^\top L \mathbf{f} \quad \text{subject to} \quad \mathbf{f}^\top D \mathbf{f} = 1, \quad \mathbf{f}^\top D \mathbf{1} = 0$$

The constraint $\mathbf{f}^\top D \mathbf{f} = 1$ prevents the trivial all-zero solution and normalizes the scale. The constraint $\mathbf{f}^\top D \mathbf{1} = 0$ excludes the trivial constant solution by requiring orthogonality to $\mathbf{1}$ in the D-weighted inner product.

Why the D-weighted inner product? Using D as the inner product matrix accounts for vertex degree: it gives higher weight to high-degree (high-frequency) words in the normalization, preventing the embedding from being dominated by rare words.

Connection to Generalized Eigenvalues

The optimization problem above is a **generalized eigenvalue problem**. By the method of Lagrange multipliers, the solution satisfies:

$$L\mathbf{f} = \lambda D \mathbf{f}$$

This is the generalized eigenproblem for the matrix pair (L, D). The solutions are the generalized eigenvectors $\mathbf{f}_1, \mathbf{f}_2, \ldots, \mathbf{f}_{|V|}$ with corresponding eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{|V|}$.

Note that $L\mathbf{f} = \lambda D\mathbf{f}$ is equivalent to $D^{-1}L\mathbf{f} = \lambda\mathbf{f}$, i.e., the standard eigenproblem for the random walk normalized Laplacian $L_{\text{rw}} = D^{-1}L = I - D^{-1}W$. The eigenvalues of L_{rw} lie in $[0, 2]$, with $\lambda_1 = 0$ corresponding to the constant eigenvector.

The first nontrivial eigenvector $\mathbf{f}_2$ (corresponding to λ_2, the smallest nonzero eigenvalue) is the **Fiedler vector**. It provides the single best one-dimensional embedding of the graph—the assignment of real numbers to vertices that minimizes variation across strong edges while excluding the trivial constant solution. It tends to split the graph into two parts, separating the two most loosely connected clusters.

Building the Embedding

For a k-dimensional embedding, we take the first k nontrivial generalized eigenvectors. Let $\mathbf{f}_2, \mathbf{f}_3, \ldots, \mathbf{f}_{k+1}$ be the generalized eigenvectors corresponding to the k smallest nonzero eigenvalues $\lambda_2 \leq \lambda_3 \leq \cdots \leq \lambda_{k+1}$.

Form the matrix:

$$\Phi = \begin{bmatrix} \mathbf{f}_2 & \mathbf{f}_3 & \cdots & \mathbf{f}_{k+1} \end{bmatrix} \in \mathbb{R}^{|V| \times k}$$

The **embedding of word** i is the i-th row of Φ:

$$\phi(w^{(i)}) = \Phi_{i,:} = (f_2(i), f_3(i), \ldots, f_{k+1}(i)) \in \mathbb{R}^k$$

Each word is now represented as a k-dimensional vector. The algorithm is summarized as:

1. **Compute context sets** $C(w)$ for each word in the vocabulary
2. **Build the similarity matrix** W using the Dice coefficient
3. **Compute the degree matrix** $D = \text{diag}(\sum_j W_{1j}, \ldots, \sum_j W_{|V|j})$
4. **Compute the Laplacian** $L = D - W$
5. **Solve the generalized eigenproblem** $L\mathbf{f} = \lambda D\mathbf{f}$ for the smallest eigenvalues
6. **Form the embedding matrix** Φ from the eigenvectors corresponding to the k smallest nonzero eigenvalues
7. **Read off word vectors** as rows of Φ

Why This Works: Intuition

The spectral embedding works because it finds the directions along which the graph varies most smoothly. The eigenvector with the smallest nonzero eigenvalue captures the "slowest"

variation—the dimension along which we can separate words while keeping similar words close together. The second eigenvector captures the next slowest variation, orthogonal to the first, and so on.

Think of it physically. Imagine the graph as a network of springs, where strong edges are stiff springs and weak edges are loose springs. The eigenvectors describe the natural modes of vibration of this spring network. The lowest-frequency modes (smallest eigenvalues) correspond to the broadest, most global structure—the major divisions in the vocabulary. Higher-frequency modes capture progressively finer distinctions.

By using the k lowest-frequency modes as our embedding dimensions, we capture the k most important axes of variation in the word similarity graph. Words that are tightly connected (similar contexts) end up with similar coordinates across all k dimensions, and hence similar embedding vectors.

Properties of Spectral Embeddings

Similarity Preservation

The central guarantee of spectral embedding is that it maps connected (similar) vertices to nearby points. More precisely, among all k-dimensional embeddings, the spectral embedding minimizes:

$$\sum_{i,j} W_{ij} \|\phi(w^{(i)}) - \phi(w^{(j)})\|^2$$

subject to the normalization constraints. This is a direct consequence of the variational characterization of eigenvalues: the first k eigenvectors of $L\mathbf{f} = \lambda D\mathbf{f}$ jointly minimize the sum of $\mathbf{f}^\top L\mathbf{f}$ over all k orthogonal directions.

In plain language: spectral embedding finds the representation where the total "stretching" across strong edges is minimized. Words connected by strong edges (high similarity) are pulled close together; words connected by weak edges are allowed to drift apart.

Nearest Neighbors and Word Similarity

Once we have the embedding, finding words similar to a given query word is straightforward: compute the nearest neighbors in the embedding space.

Given a query word w, its m nearest neighbors are:

$$\text{NN}_m(w) = \underset{S \subset V,\, |S|=m}{\arg\min} \sum_{w' \in S} \|\phi(w) - \phi(w')\|^2$$

In practice, we compute Euclidean distances from $\phi(w)$ to all other word vectors and return the m closest.

For example, querying the word "made" in an embedding computed from a large corpus might return neighbors like: *built, created, produced, constructed, formed, designed, developed, generated*—a cluster of words that share the "past tense of creation verbs" pattern. This is not because the algorithm was told anything about verb morphology; it emerges purely from distributional context.

Going further, the embeddings can reveal morphological and syntactic structure. If we query past-tense verb forms, their neighborhoods tend to contain other past-tense forms. If we query plural nouns, their neighborhoods tend to contain other plural nouns. The grammatical categories that linguists identify through careful analysis emerge naturally from the statistics of word usage.

Clustering in the Embedding Space

The embedding vectors can serve as input to standard clustering algorithms (k-means, hierarchical clustering, DBSCAN, etc.) to discover groups of related words. Because spectral embedding preserves graph structure, the clusters in the embedding space correspond to densely connected regions of the word similarity graph—groups of words that share many contexts.

This provides a data-driven approach to word categorization: instead of relying on manually constructed lexicons or hand-written rules, we let the distributional statistics in the corpus determine which words group together.

Visualization

Spectral embeddings with $k = 2$ or $k = 3$ can be directly visualized as scatter plots, providing a "map" of the vocabulary. Even for higher-dimensional embeddings, dimensionality reduction techniques like t-SNE or PCA can project the vectors to 2D for visualization.

Such visualizations can serve as a "megascope"—a tool for visualizing and exploring large amounts of linguistic data simultaneously. By examining how words cluster, which words are

nearby, and how neighborhoods connect, linguists and NLP practitioners can form hypotheses about language structure, compare different corpora, or even compare different languages.

Worked Example

A Small Corpus

Consider the following toy corpus of 5 sentences:

1. "the cat sat on the mat"
2. "the dog sat on the rug"
3. "a cat chased a dog"
4. "the bird sat on the branch"
5. "a dog chased a bird"

We'll use a context window of size 1 (immediate neighbors only) and only single-word context features for simplicity.

Computing Context Sets

For each word, we collect the set of words appearing immediately to its left or right across the corpus:

Word	**Left contexts**	**Right contexts**	$C(w)$
cat	the, a	sat, chased	{the, a, sat, chased}
dog	the, a	sat, chased	{the, a, sat, chased}
bird	the, a	sat, chased	{the, a, sat, chased}
sat	cat, dog, bird	on	{cat, dog, bird, on}
mat	the	(end)	{the}
rug	the	(end)	{the}
branch	the	(end)	{the}
chased	cat, dog	a	{cat, dog, a}

Computing Similarities

Notice that "cat," "dog," and "bird" have identical context sets: $C(\text{cat}) = C(\text{dog}) = C(\text{bird}) = \{\text{the, a, sat, chased}\}$. Therefore:

$$D(\text{cat}, \text{dog}) = 2 \times \frac{|C(\text{cat}) \cap C(\text{dog})|}{|C(\text{cat})| + |C(\text{dog})|} = 2 \times \frac{4}{4+4} = 1.0$$

Similarly, "mat," "rug," and "branch" all have $C = \{\text{the}\}$, so they are maximally similar to each other:

$$D(\text{mat}, \text{rug}) = 1.0$$

But "cat" and "mat" share only {the}:

$$D(\text{cat}, \text{mat}) = 2 \times \frac{1}{4+1} = 0.4$$

Even in this tiny example, the Dice coefficient captures meaningful structure: animate nouns (cat, dog, bird) cluster together, and location nouns (mat, rug, branch) cluster together.

The Resulting Embedding

After constructing the full similarity matrix and computing the spectral embedding, the animate nouns would receive nearly identical embedding vectors (they have identical context distributions), and the location nouns would similarly cluster together. The spectral embedding would place these two clusters in different regions of the embedding space, reflecting the two distinct distributional patterns.

Connection to Other Embedding Methods

Count-Based Methods

Spectral embedding belongs to a family of **count-based** embedding methods that derive word vectors from co-occurrence statistics. Other members include:

Latent Semantic Analysis (LSA). Constructs a word-document or word-context co-occurrence matrix and applies Singular Value Decomposition (SVD) to obtain low-rank approximations. The resulting left singular vectors serve as word embeddings. LSA operates on raw counts (possibly weighted by TF-IDF), while spectral embedding first converts counts to a similarity measure and then applies spectral decomposition of the graph Laplacian.

Pointwise Mutual Information (PMI) methods. Replace raw co-occurrence counts with PMI scores before applying SVD. Levy and Goldberg (2014) showed that certain neural embedding methods implicitly factor a shifted PMI matrix.

GloVe (Pennington et al., 2014). Despite often being grouped with neural methods, GloVe is fundamentally a matrix factorization approach. It constructs a global word-word co-occurrence matrix and learns embeddings by factorizing the log-count matrix through a weighted least-squares objective. The "vectors" in GloVe are the factors of this decomposition, placing it squarely in the count-based tradition alongside LSA and PMI methods.

Neural Methods

The neural embedding methods that revolutionized NLP in the 2010s—**Word2Vec** (Mikolov et al., 2013), **ELMo** (Peters et al., 2018), and **fastText** (Bojanowski et al., 2017)—also build on the distributional hypothesis, but they learn embeddings by training neural networks on co-occurrence prediction tasks rather than through explicit matrix decomposition. ELMo goes further by producing *contextualized* embeddings: rather than assigning a single vector to each word type, it uses a deep bidirectional LSTM to generate representations that vary with the surrounding sentence, capturing polysemy and context-dependent meaning.

Despite the very different algorithms, the underlying principle is the same: words that appear in similar contexts should get similar vectors. We will develop these neural methods in detail in future articles.

What Spectral Embedding Offers

Spectral embedding has several distinctive properties:

Mathematical transparency. The algorithm is entirely linear-algebraic: build a matrix, compute eigenvalues. There are no training loops, no hyperparameters for learning rate or batch size, no risk of poor convergence. The solution is unique (up to trivial rotations and sign flips).

Theoretical guarantees. Spectral methods have well-understood optimality properties from spectral graph theory. The embedding provably minimizes the graph cut objective, connecting word embedding to a rich mathematical literature.

Interpretability. Each dimension of the embedding corresponds to a specific eigenvector of the Laplacian, which can be examined to understand what linguistic distinction it captures.

The main limitations are computational: forming the full $|V| \times |V|$ similarity matrix and computing its eigendecomposition is expensive for large vocabularies. This is where neural methods gain a practical advantage—they can train on the corpus directly in an online fashion, without ever forming the full co-occurrence matrix.

Summary

Word embeddings transform discrete vocabulary items into dense vectors in a continuous space, enabling geometric reasoning about word similarity and providing natural inputs for machine learning models.

The distributional hypothesis provides the foundation: words that appear in similar contexts have similar meanings. By collecting context features from a corpus, we can quantify word similarity using set overlap measures like the Dice coefficient.

Spectral embedding converts the resulting similarity matrix into a word graph and applies spectral decomposition of the graph Laplacian to find low-dimensional vector representations:

- Construct the similarity matrix W from distributional context overlap
- Compute the graph Laplacian $L = D - W$
- Solve the generalized eigenvalue problem $L\mathbf{f} = \lambda D\mathbf{f}$
- Use the eigenvectors corresponding to the smallest nonzero eigenvalues as embedding dimensions
- Read off word vectors as rows of the eigenvector matrix

The resulting embeddings place similar words near each other in the vector space, enabling nearest-neighbor queries for word similarity, clustering for word categorization, and visualization for linguistic exploration.

The quadratic form $\mathbf{f}^\top L\mathbf{f} = \frac{1}{2}\sum_{i,j} W_{ij}(f_i - f_j)^2$ is the key mathematical fact: the Laplacian measures how much a function varies across edges, and minimizing this quantity (via eigendecomposition) yields embeddings that keep similar words close together.

This spectral approach illustrates the core ideas that underlie all word embedding methods—distributional context, similarity measurement, and dimensionality reduction—in a mathematically transparent framework. The neural embedding methods covered in subsequent

articles achieve the same goals through different computational means, but the conceptual foundation remains the same: you shall know a word by the company it keeps.